



IIC2343 - Arquitectura de Computadores (II/2026)

Interrogación 1

Pauta de evaluación

Pregunta 1: Representación de números enteros (6 ptos.)

- (a) (2 ptos.) Suponga que tiene dos representaciones binarias distintas: una representación de 8 bits para números positivos (incluido el cero); y una representación de 8 bits para números enteros (*i.e.* positivos, negativos y el cero). ¿Cuál de las dos representaciones permite representar una mayor cantidad de números distintos? Justifique.

Solución: Ninguna. En ambas se pueden representar $2^8 = 256$ números distintos. La diferencia radica en el rango: la representación de números positivos posee el rango $[0, 255]$, mientras que el de la representación de números enteros corresponde a $[-128, 127]$. En ambos casos, la cantidad de valores distintos que se representa es la misma. Se otorga **1 pto.** por señalar correctamente que ambas poseen la misma cantidad de números distintos, y **1 pto.** por justificación.

- (b) (2 ptos.) Escriba la representación en decimal del resultado de la operación $0xFFFF + 0xFFFE$. El resultado debe interpretarse como binario con signo de 16 bits.

Solución: Si ambos operandos se representan como binarios con signo, entonces la operación realizada corresponde a $-1 + -2 = -3$. Al no generarse *overflow*, este resultado se puede representar como $0xFFFFD$. Cabe destacar que no se espera que realicen la suma en una base en particular, sino que sean capaces de deducir el resultado de la operación y expresarlo en base decimal. Se otorga **1 pto.** por obtener correctamente el resultado de la operación, ya sea en base binaria o hexadecimal, 1 **1 pto.** por representarlo correctamente en base decimal con signo.

- (c) (2 ptos.) Indique y explique en qué tipos de representación binaria de N bits es correcto asumir que, si $A + B$ genera un acarreo, entonces la operación generó un *overflow*.

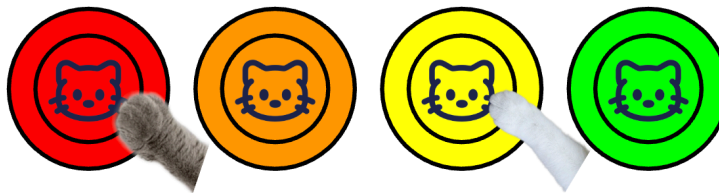
Solución: El acarreo implica *overflow* en representaciones binarias sin signo, *i.e.* donde solo se representan números positivos. El motivo radica en el hecho de que, al sumar y generar un acarreo, esto indica que el resultado requiere de 1 bit adicional para ser representado, lo que no es posible dado el límite de N bits. Si se representaran tanto positivos como negativos, esta sentencia ya no sería correcta. Si se denota c como el acarreo de la suma, se tienen los siguientes contraejemplos:

- $1 + (-1) = 00000001b + 11111111b = (1)_c 00000000b = 0 \rightarrow c = 1$, pero no hubo *overflow*.
- $1 + 127 = 00000001b + 01111111b = (0)_c 10000000 = -128 \rightarrow c = 0$, pero hubo *overflow*.

En los mismos ejemplos antes descritos se puede observar que, si los operandos y el resultado se interpretaran como números positivos, entonces sí habría una correlación entre el acarreo generado y la existencia de *overflow*. Se otorga **1 pto.** por señalar correctamente el tipo de representación donde el acarreo se correlaciona con *overflow*, y **1 pto.** por justificación.

Pregunta 2: Circuitos digitales y almacenamiento (6 ptos.)

- (a) (3 ptos.) Suponga que vive con dos gatos y, con el fin de mantenerlos contentos, diseña un sistema de cuatro botones denotados como B_i donde $B_i = 1$ indica que el botón i está siendo presionado:



Los gatos están entrenados para presionarlos de la siguiente forma: el primero presiona B_0 si tiene hambre y B_1 si tiene sed; mientras que el segundo presiona B_2 si tiene hambre y B_3 si tiene sed. Estos botones se conectan a tres dispensadores D_i : D_0 , de comida; D_1 , de agua; y D_2 , de churu. Estos se activan de la siguiente forma:

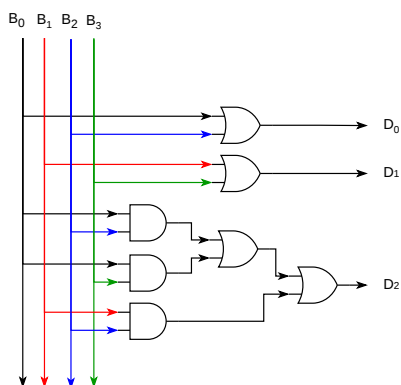
- $D_0 = 1$ si al menos uno de los dos gatos tiene hambre.
- $D_1 = 1$ si al menos uno de los dos gatos tiene sed.
- $D_2 = 1$ si ambos gatos tienen hambre; o si un gato tiene hambre y el otro tiene sed. **No se activa si un gato presiona sus dos botones y el otro no presiona ninguno.**

A partir de este contexto, diseñe y dibuje para cada salida D_i un circuito combinacional que reciba las señales de entrada B_i y compute si el contenido debe ser dispensado (1) o no (0). Exprese claramente las funciones lógicas utilizadas para cada circuito.

Solución: Si bien este circuito se puede diseñar a partir de *minterms* y *maxterms*, la expresión lógica de cada salida D_i se puede expresar de la siguiente forma:

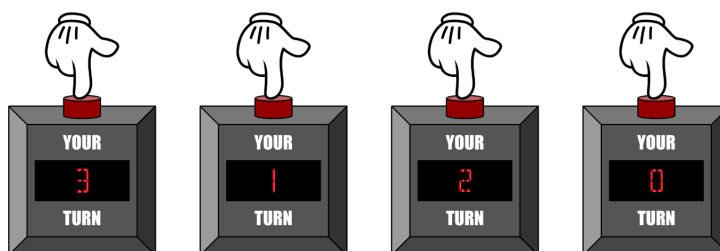
- $D_0 = B_0 \vee B_2 \rightarrow$ El primer gatito tiene hambre o el segundo gatito tiene hambre.
- $D_1 = B_1 \vee B_3 \rightarrow$ El primer gatito tiene sed o el segundo gatito tiene sed.
- $D_2 = (B_0 \wedge B_2) \vee (B_0 \wedge B_3) \vee (B_1 \wedge B_2) \rightarrow$ Ambos gatitos tienen hambre; el primero tiene hambre y el segundo sed; o el primero tiene sed y el segundo hambre.

A partir de estas expresiones, se puede diseñar el circuito para cada una de las salidas:



Es importante destacar que se evaluará la correctitud del circuito diseñado, no que sea igual al de esta pauta. Se otorga **1 pto.** por el circuito correctamente diseñado de cada dispensador. En cada caso se otorga la mitad del puntaje si existe, como máximo, una combinación de señales de entrada que generen una salida incorrecta.

- (b) (3 ptos.) Para aplicar los contenidos del curso, decide implementar un **indicador de turno** para juegos de mesa de cuatro jugadores. En particular, desea la siguiente configuración:



Esto indica que el primer turno corresponde al jugador 3; luego al jugador 1; posteriormente al jugador 2; y finalmente al jugador 0. Para esto debe diseñar, utilizando **solo** compuertas lógicas, *enablers*, multiplexores y flip-flops, un contador secuencial de 2 bits que se actualiza en cada flanco de subida a partir de una señal *Push* según las siguientes condiciones:

- El contador parte en 3 ($Q_1Q_0 = 11$). Asuma que los flip-flops del contador parten en 1.
- El contador se actualiza en el flanco de subida si, y solo si $Push = 1$.

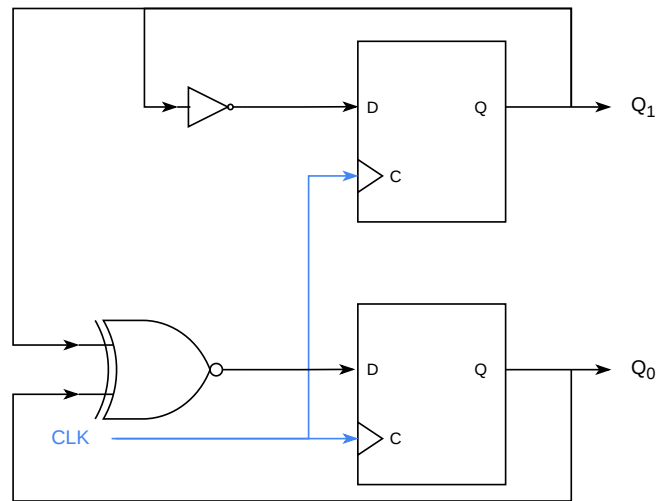
Puede asumir que su contador hace *overflow* en caso de actualizarse estando en su mínimo valor, *i.e.* asuma que Q_1Q_0 pasa de 00 a 11 al modificarse con $Push = 1$.

Hint: Para optar a la mitad del puntaje, primero diseñe un contador que siga la secuencia señalada, y luego incorpore el uso de la señal *Push*. Puede realizar dos dibujos separados.

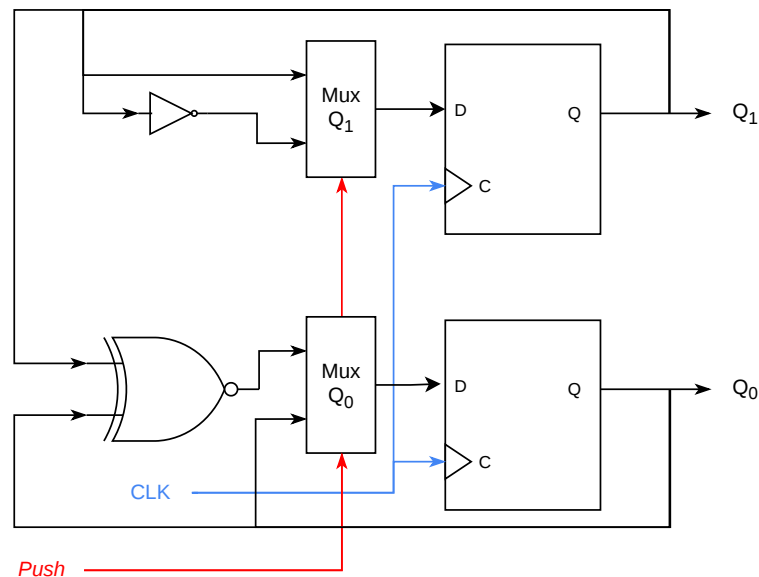
Solución: Si denotamos por $Q_1^tQ_0^t$ el estado del contador en un ciclo t de la señal de control CLK y $Q_1^{t+1}Q_0^{t+1}$ su estado en el próximo ciclo, se puede elaborar la siguiente tabla de verdad:

Q_1^t	Q_0^t	Q_1^{t+1}	Q_0^{t+1}
0	0	1	1
0	1	1	0
1	0	0	0
1	1	0	1

Se deduce que $Q_0^{t+1} = \overline{(Q_0^t \oplus Q_1^t)}$ y $Q_1^{t+1} = \overline{Q_1^t}$. Es decir, Q_0 será igual a Q_0 XNOR Q_1 , mientras que Q_1 será igual a NOT Q_1 . Haciendo uso de flip-flops y las compuertas antes señaladas, se llega al siguiente diagrama:



Ahora, esta es una solución intermedia porque no contempla el caso en el que $Push = 0$, donde el contador debería mantener su estado. Para ello, agregaremos un multiplexor para cada señal Q_i que seleccionará los estados originales del contador si $Push = 0$ y la salida de las compuertas descritas anteriormente si $Push = 1$:



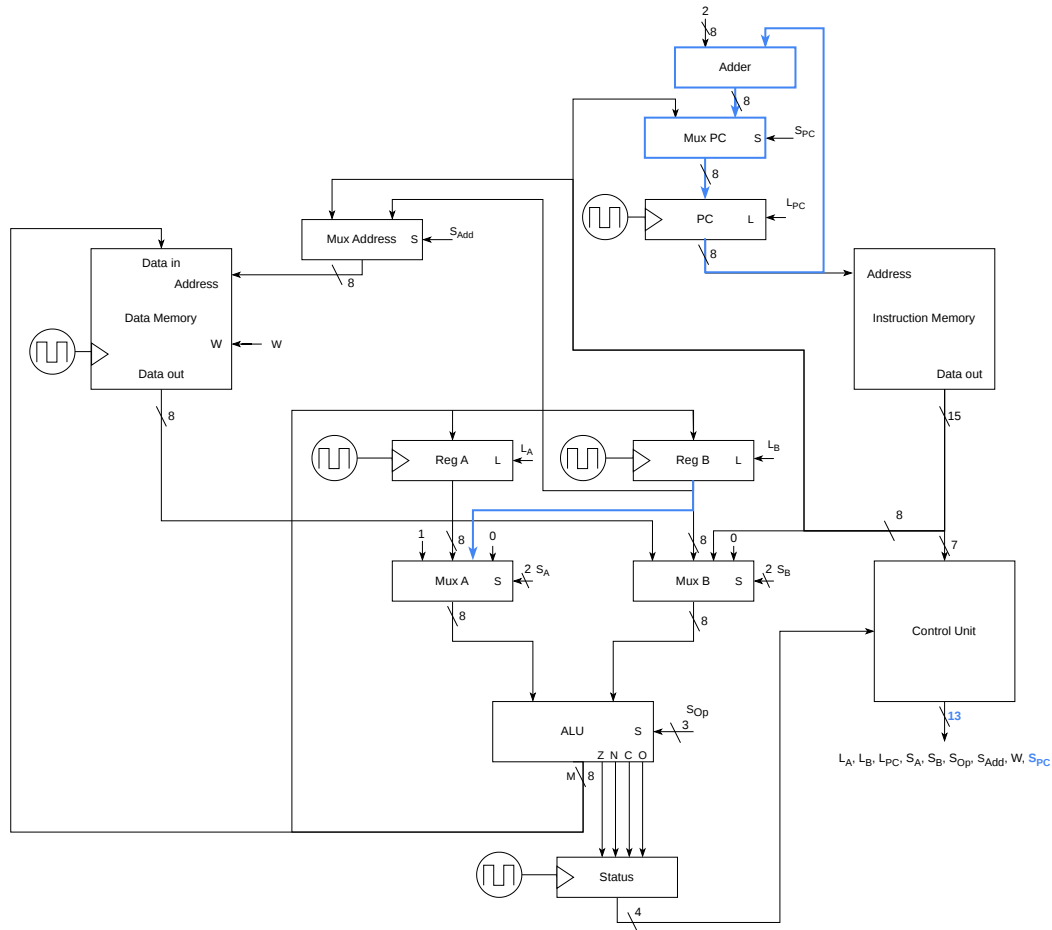
Es importante notar que se evaluará la correctitud del contador diseñado, no que sea igual al de esta pauta. Se otorgan **1.5 ptos.** por diseñar correctamente un contador de 2 bits con las características descritas en el enunciado, y **1.5 ptos.** por actualizar su valor correctamente a partir de la señal $Push$. Por cada criterio, se otorga la mitad del puntaje si existe, como máximo, un error de implementación.

Pregunta 3: Computador básico con saltos (6 ptos.)

En los ítems (a), (b) y (c), se le describirán instrucciones que deberá implementar en el computador básico a partir del diagrama adjunto. Para cada una de ellas, debe incluir la combinación **completa** de señales que las ejecutan. En las señales de carga/escritura, debe indicar si se activan (1) o no (0); en las señales de selección, debe indicar el **nombre** de la entrada escogida (“-” si no afecta). Debe realizar todas las modificaciones de *hardware* en un solo diagrama. Además, puede implementar instrucciones sin modificaciones si es que la arquitectura se lo permite, indicando las señales de control necesarias. **Cada instrucción debe ejecutarse en un ciclo.**

- (a) (1 pto.) **NEG** (Dir). Almacena en Mem[Dir] su inverso aditivo, *i.e.* $\text{Mem}[\text{Dir}] = -\text{Mem}[\text{Dir}]$.
- (b) (1 pto.) **SHL** B,B. Almacena en el registro B el resultado de realizar un *shift left* sobre B.
- (c) (1 pto.) **SKIP**. Omite la siguiente instrucción, *i.e.* carga en el registro PC el valor $\text{PC} + 2$.

Solución: A continuación, se muestra el diagrama con las conexiones necesarias para ejecutar todas las instrucciones solicitadas, incluyendo una descripción de cada conexión añadida y lo que habilita.



- Para la instrucción **NEG** (Dir), no se hacen modificaciones. El valor deseado se puede computar con la microarquitectura original ejecutando la operación $0 - \text{DOUT}$ en la ALU.
- Para la instrucción **SHL B,B**, se conecta la salida del registro B con el multiplexor Mux A. Esto permite poder seleccionar B como operando de la ALU en la operación *shift left*.
- Para la instrucción **SKIP**, es necesario agregar dos cosas: en primer lugar, un sumador de 8 bits que compute la suma entre PC y un 2 (dirección a la que se desea saltar); y en segundo lugar un multiplexor Mux PC con señal de selección S_{PC} para escoger entre PC y $PC + 2$ como valor a ser cargado. Esto hace que la salida de la **Control Unit** incremente a 13 bits.

A continuación, la tabla de señales resultante:

Instrucción	L_A	L_B	L_{PC}	W	S_A	S_B	S_{OP}	S_{Add}	S_{PC}
NEG (Dir)	0	0	0	1	ZERO	DOUT	SUB	LIT	-
SHL B,B	0	1	0	0	B	-	SHL	-	-
SKIP	0	0	1	0	-	-	-	-	PC+2

Por cada instrucción, se otorgan:

- **0.5 ptos.** por *hardware* correctamente modificado; **0.3 ptos.** si existe **un solo error** de implementación; **0.1 ptos.** si existen **solo dos errores**; y **0 ptos.** en otro caso.
- **0.5 ptos.** por entregar una combinación de señales correcta; **0.3 ptos.** si existe **un solo error**; **0.1 ptos.** si existen **solo dos errores**; y **0 ptos.** en otro caso. Si una instrucción se define en más de un ciclo, tampoco se otorga puntaje en este criterio. Si la instrucción se implementa correctamente sin modificación de *hardware*, se otorga el puntaje completo.

- (d) (3 ptos.) Los programas (1) y (2) están escritos en el Assembly del computador básico del curso, con la diferencia de que (2) usa las instrucciones implementadas en los ítems anteriores. Para cada programa, indique los valores de los registros A y B al finalizar la ejecución. Si no incluye desarrollo o no justifica cómo llegó a los valores obtenidos, **no se otorgará puntaje**.

(1)

```
DATA:
    var1 3
    var2 5
    res 0
CODE:
    MOV A, (var1)
    MOV B, var2
    XOR A, B
    XOR B, A
    XOR A, B
    ADD (res)
```

(2)

```
DATA:
    arr 2
        7
        4
    len 3
    i 0
    temp 0
CODE:
    loop:
        MOV A, arr
        ADD A, (i)
        MOV B, A
        MOV A, (B)
        MOV B, A
        AND A, 1
        JGT continue
        NEG (len)
        SKIP
    continue:
        SHL B, B
        MOV A, arr
        ADD A, (i)
        MOV (temp), A
        MOV A, B
        MOV B, (temp)
        MOV (B), A
        NEG (B)
    next:
        INC (i)
        MOV A, (i)
        CMP A, (len)
        JGE end
        JMP loop
    end:
        MOV B, (arr)
```

Solución: El fragmento de código (1) realiza un *swap* entre los registros A y B, con la salvedad de que A guarda el valor de `var1` y B guarda el valor de la **dirección** de `var2`. El fragmento de código (2), por otra parte, tiene como espíritu original transformar cada elemento `arr[i]` en $-2 \cdot \text{arr}[i]$. No obstante, como no se ejecuta el salto `JGT continue`, la ejecución de la instrucción `NEG(len)` genera que se actualice la variable `len` a -3, haciendo que `loop` se ejecute una sola vez y no por cada elemento del arreglo, y que el registro B nunca se duplique por omitir `SHL B, B` a través de la ejecución de `SKIP`. A continuación, se muestra el resultado de la ejecución para ambos fragmentos:

(1)

```
DATA:
var1 3      // MEM[0]
var2 5      // MEM[1]
res 0       // MEM[2]
CODE:
MOV A, (var1) // A = 3
MOV B, var2   // B = dir. var2 = 1
XOR A, B      // A = (A XOR B)
XOR B, A      // B = A XOR B = (A XOR B) XOR B = A XOR (B XOR B) = A XOR 0 = A = 3
XOR A, B      // A = A XOR B = (A XOR B) XOR A = (A XOR A) XOR B = 0 XOR B = B = 1
ADD (res)     // res = A + B = 1 + 3 = 4
```

(2)

```
DATA:
arr 2      // MEM[0]
    7      // MEM[1]
    4      // MEM[2]
len 3      // MEM[3]
i 0        // MEM[4]
temp 0     // MEM[5]
CODE:
loop:
MOV A, arr  // A = 0
ADD A, (i)   // A = 0 + 0 = 0
MOV B, A     // B = 0
MOV A, (B)   // A = MEM[B] = MEM[0] = 2
MOV B, A     // B = 2
AND A, 1     // A = 00000010 AND 00000001 = 0, flags Z = 1 y N = 0
JGT continue // Salta si Z == 0 y N == 0 -> No salta
NEG (len)    // len = -3
SKIP        // Se salta SHL B, B
continue:
SHL B, B     // No se ejecuta por la ejecución de SKIP
MOV A, arr   // A = 0
ADD A, (i)   // A = 0 + 0 = 0
MOV (temp), A // temp = 0
MOV A, B     // A = 2
MOV B, (temp) // B = 0
MOV (B), A   // MEM[0] = 2 -> arr[0] = 2
NEG (B)      // MEM[0] = -2 -> arr[0] = -2
next:
INC (i)      // i = 1
MOV A, (i)   // A = 1
CMP A, (len) // 1 - -3 = 1 + 3 = 4, flags Z = 0 y N = 0
JGE end     // Salta si N == 0 -> Salta a end
JMP loop     // No se alcanza a ejecutar
end:
MOV B, (arr) // B = arr[0] = -2
```

Para cada fragmento, se otorgan **0.75 ptos.** por registro correctamente computado siempre que su cómputo esté respaldado por desarrollo o justificación. Si no se llega a ningún resultado final correcto por **error de arrastre**, se otorgan **0.5 ptos.** en el fragmento.